Developing Underworld using Python Virtual Environments

Romain Beucher[1]

[1] Australian National University

DOI https://doi.org/10.6084/m9.figshare.33193515

Keywords Documentation, development, Python/Jupyter

The Underworld Community has been supporting usage of Docker containers for developing and running Underworld powered scientific models. They provide a good option to control the running environment (dependencies).

Docker container can be used for developing the code base but creating a suitable development workflow has proven challenging so far and I have opted for a more simple solution involving native builds of PETSc and Python Virtual Environment.

In the following I describe how I install Underworld (v2.12) in a Python virtual environment. This is by no way perfect and others may have a different approach, I always welcome suggestions!

0.a. PETSc

The main dependency that we use is PETSc (the Portable, Extensible Toolkit for Scientific Computation). PETSc installation is not complicated but can be time-consuming depending on how many External Packages (Solvers etc.) are required. I suggest reading my other blogpost to learn what's needed and how to install PETSc on your machine.

PETSc tends to evolve quite fast and we thus need to make sure that we keep up to date with the changes introduced with each new version. We also need to be able to test for some sort of back-compatibility.
My solution to this problem has been to compile different versions and configuration of PETSc on my machine. They are all located in the `/opt` directory and I can switch easily between versions as needed.

Here is a snapshot of my `/opt/` directory:

```
3.13.6  3.14.6  3.15.5  3.16.1
```

0.b. Python Virtual Environment

Python Virtual Environment provide an easy way to control your python packages stack. It is not the only way to achieve this, some other tools such as Conda and Module files can do the same thing and actually extend

the idea to other non-python-based applications.
So here again all roads lead to Rome and you just need to find what works for you.

To use virtual environments you first need to install the appropriate package on your machine. Note that it is good practice to use the package provided by your LINUX distribution.

On Arch Linux we use:

```
sudo pacman -Syy
sudo pacman -S python-virtualenv
```

on Ubuntu

```
sudo apt-get update
sudo apt-get install -y python3-venv
```

Virtual Environment are created using:

```
python -m venv --name my_underworld_env
```

The virtual environment is then activated by sourcing the activate script as follow:

```
source my_underworld_env/bin/activate
```

0.c. Handling PETSc

The `activate` script that you source when activating a Python virtual environment contains a set of instructions that set the `PATH` and `PYTHONHOME` environment variables.
The trick to handle PETSc is to also let the `activate` script to manage the `PETSC_DIR` environment variable. The `PETSC_DIR` variable should point to the PETSc version you want to use with the Python virtual environment.

In the following I am modifying the script so that it creates:

- PETSC_DIR (that points to the PETSc built I want to use)
- PYTHONPATH (points to `$PETSC_DIR/lib`)
- PATH (points to `$PETSC_DIR/bin` where I have `mpirun` etc.)

I also modify the `deactivate` function so that variables are reset when we deactivate the virtual environment.

```
# This file must be used with "source bin/activate" *from bash*
# you cannot run it directly

deactivate () {
    # reset old environment variables
    if [ -n "${_OLD_VIRTUAL_PATH:-}" ] ; then
        PATH="${_OLD_VIRTUAL_PATH:-}"
        export PATH
        unset _OLD_VIRTUAL_PATH
    fi
```

```
    if [ -n "${_OLD_VIRTUAL_PYTHONHOME:-}" ] ; then
        PYTHONHOME="${_OLD_VIRTUAL_PYTHONHOME:-}"
        export PYTHONHOME
        unset _OLD_VIRTUAL_PYTHONHOME
    fi

    if [ -n "${_OLD_PYTHONPATH:-}" ] ; then
        PYTHONPATH="${_OLD_PYTHONPATH:-}"
        export PYTHONPATH
        unset _OLD_PYTHONPATH
    fi
    if [ -n "${_OLD_PETSC_DIR:-}" ] ; then
        PETSC_DIR="${_OLD_PETSC_DIR:-}"
        export PETSC_DIR
        unset _OLD_PETSC_DIR
    fi

    # This should detect bash and zsh, which have a hash command that must
    # be called to get it to forget past commands.  Without forgetting
    # past commands the $PATH changes we made may not be respected
    if [ -n "${BASH:-}" -o -n "${ZSH_VERSION:-}" ] ; then
        hash -r 2> /dev/null
    fi

    if [ -n "${_OLD_VIRTUAL_PS1:-}" ] ; then
        PS1="${_OLD_VIRTUAL_PS1:-}"
        export PS1
        unset _OLD_VIRTUAL_PS1
    fi

    unset VIRTUAL_ENV
    if [ ! "${1:-}" = "nondestructive" ] ; then
    # Self destruct!
        unset -f deactivate
    fi
}

# unset irrelevant variables
deactivate nondestructive

VIRTUAL_ENV="/home/romain/my_underworld_venv"
export VIRTUAL_ENV

_OLD_VIRTUAL_PATH="$PATH"
PATH="/opt/petsc/3.16.1/bin:$VIRTUAL_ENV/bin:$PATH"
export PATH

# unset PYTHONHOME if set
# this will fail if PYTHONHOME is set to the empty string (which is bad anyway)
# could use `if (set -u; : $PYTHONHOME) ;` in bash
if [ -n "${PYTHONHOME:-}" ] ; then
    _OLD_VIRTUAL_PYTHONHOME="${PYTHONHOME:-}"
    unset PYTHONHOME
fi

if [ -z "${VIRTUAL_ENV_DISABLE_PROMPT:-}" ] ; then
```

```
    _OLD_VIRTUAL_PS1="${PS1:-}"
    PS1="(uw2_venv) ${PS1:-}"
    export PS1
fi

# This should detect bash and zsh, which have a hash command that must
# be called to get it to forget past commands.  Without forgetting
# past commands the $PATH changes we made may not be respected
if [ -n "${BASH:-}" -o -n "${ZSH_VERSION:-}" ] ; then
    hash -r 2> /dev/null
fi

_OLD_PYTHONPATH="$PYTHONPATH"
PYTHONPATH=/opt/petsc/3.16.1/lib
export PYTHONPATH

_OLD_PETSC_DIR="$PETSC_DIR"
PETSC_DIR=/opt/petsc/3.16.1
export PETSC_DIR
```

Now sourcing the activate script sets `PETSC_DIR` `PATH` and `PYTHONPATH` to the appropriate values. Running `deactivate` takes us back to normal. We are ready to install Underworld and the python packages we need!

0.d.

0.d.i. *Installing Underworld in development mode:*

Underworld requires some building dependencies.

Ubuntu::

```
apt-get update -qq
apt-get install \
        build-essential \
        pkg-config \
        python3-dev \
        cmake \
        make \
        swig \
        libxml2-dev \
        g++
```

0.d.ii. *Installing h5py:*

After activating the Python virtual Environment so that the install process uses the appropriate version og HDF5 and H5PCC

```
source my_underworld_venv/bin/activate

CC=h5pcc HDF5_MPI="ON" HDF5_DIR=${PETSC_DIR} pip3 install --no-cache-dir --no-binary=h5py
git+https://github.com/h5py/h5py@master
```

Get the current development branch of Underworld from GitHub

```
source my_underworld_venv/bin/activate

git clone git@github.com:underworldcode/underworld2.git
cd underworld2
git checkout development
```

Then build Underworld

```
# From the underworld directory
pip install -vvv -e .
```

You should now have a Python virtual environment with a working version of Underworld

1. Conclusion

Python Virtual Environment can be useful for Underworld development. You can easily configure them so that they point to a specific version of PETSc on your machine.
This make possible to test the code against different versions of PETSc, run tests and try different optimization options or configurations of the software stack.